

DSA + AI Interview Questions & Proper Answers

Complete interview-ready study set compiled from the uploaded AI question screenshots, plus a core DSA set.

Contents

Part 1 — DSA: 30 core interview questions

Part 2 — AI/LLM: 50 questions covering LLM fundamentals, RAG & retrieval, prompt engineering, inference/optimization, and production LLM engineering.

Part 1 — DSA Interview Questions

1. What is a data structure?

Answer: A data structure is a way of organizing and storing data so it can be accessed and modified efficiently. Examples include arrays, linked lists, stacks, queues, trees, graphs, and hash tables.

2. What is an algorithm?

Answer: An algorithm is a finite sequence of well-defined steps used to solve a problem or perform a computation.

3. What is Big-O notation?

Answer: Big-O describes how an algorithm's time or space requirement grows as input size n increases. For example, $O(1)$ is constant, $O(\log n)$ logarithmic, $O(n)$ linear, and $O(n^2)$ quadratic.

4. Array vs linked list?

Answer: An array stores elements contiguously and supports $O(1)$ indexed access, but insertion/deletion in the middle is usually $O(n)$. A linked list uses nodes connected by pointers; access is $O(n)$, while insertion/deletion can be $O(1)$ when the node position is already known.

5. What is a stack?

Answer: A stack follows LIFO: Last In, First Out. Main operations are push, pop, and peek, normally $O(1)$. It is used in recursion, undo operations, parsing, and DFS.

6. What is a queue?

Answer: A queue follows FIFO: First In, First Out. Enqueue adds at the rear and dequeue removes from the front. It is used in scheduling, buffering, BFS, and producer-consumer systems.

7. Stack vs queue?

Answer: A stack removes the most recently inserted item, while a queue removes the earliest inserted item. Stack is LIFO; queue is FIFO.

8. What is a hash table?

Answer: A hash table stores key-value pairs using a hash function to map keys to buckets. Average lookup, insertion, and deletion are $O(1)$, although collisions can make the worst case $O(n)$.

9. How are hash collisions handled?

Answer: Common methods are separate chaining, where each bucket stores multiple entries, and open addressing, where another empty slot is searched using probing such as linear or quadratic probing.

10. What is binary search?

Answer: Binary search repeatedly halves a sorted search range. Its time complexity is $O(\log n)$ and its iterative space complexity is $O(1)$.

11. What is the requirement for binary search?

Answer: The data must be ordered according to the comparison being used. Without a sorted or otherwise monotonic search space, standard binary search is not valid.

12. What is recursion?

Answer: Recursion is when a function calls itself on a smaller subproblem. It needs a base case to stop and uses the call stack, so deep recursion can cause stack overflow.

13. What is a linked list?

Answer: A linked list is a sequence of nodes where each node stores data and a link to another node. Variants include singly, doubly, and circular linked lists.

14. How do you detect a cycle in a linked list?

Answer: Use Floyd's tortoise-and-hare algorithm: move one pointer one step and another two steps. If they meet, a cycle exists. Time is $O(n)$ and extra space is $O(1)$.

15. How do you reverse a linked list?

Answer: Maintain previous, current, and next pointers. Save current.next, point current.next to previous, then advance the pointers. This takes $O(n)$ time and $O(1)$ extra space.

16. What is a tree?

Answer: A tree is a hierarchical, connected, acyclic data structure with a root and child nodes. It is commonly used for file systems, indexes, and hierarchical data.

17. What is a binary tree?

Answer: A binary tree is a tree in which each node has at most two children, commonly called left and right.

18. What is a BST?

Answer: A Binary Search Tree maintains values so that the left subtree contains smaller values and the right subtree contains larger values, under the chosen duplicate policy. Average search/insert/delete can be $O(\log n)$, but an unbalanced BST can degrade to $O(n)$.

19. What are tree traversals?

Answer: DFS traversals include preorder (root-left-right), inorder (left-root-right), and postorder (left-right-root). BFS is level-order traversal. Inorder traversal of a BST produces sorted order.

20. What is a heap?

Answer: A heap is a complete binary tree satisfying a heap property. In a min-heap the smallest value is at the root; in a max-heap the largest is at the root. Insert and remove-root are $O(\log n)$.

21. What is a priority queue?

Answer: A priority queue removes the element with the highest or lowest priority rather than simply the oldest element. A heap is a common implementation.

22. What is a graph?

Answer: A graph consists of vertices and edges. It may be directed or undirected, weighted or unweighted, and is used to model networks, dependencies, maps, and relationships.

23. BFS vs DFS?

Answer: BFS explores level by level using a queue and is useful for shortest paths in unweighted graphs. DFS explores deeply using recursion or a stack and is useful for connectivity, cycle detection, and backtracking.

24. What is Dijkstra's algorithm?

Answer: Dijkstra finds shortest paths from a source in a graph with non-negative edge weights. With a binary heap and adjacency list, a common complexity is $O((V+E) \log V)$.

25. Dijkstra vs BFS?

Answer: BFS gives shortest paths in unweighted graphs. Dijkstra handles non-negative weighted edges. If all edge weights are equal, BFS is usually simpler.

26. What is dynamic programming?

Answer: Dynamic programming solves overlapping subproblems by storing their results. It usually combines optimal substructure with memoization (top-down) or tabulation (bottom-up).

27. Memoization vs tabulation?

Answer: Memoization is top-down recursion plus caching. Tabulation is bottom-up iteration that fills a table from smaller states to larger states. Both avoid repeated computation.

28. What is a greedy algorithm?

Answer: A greedy algorithm repeatedly chooses the locally best option. It is correct only when the problem has the required greedy-choice property and optimal substructure; examples include Kruskal and Prim for MST.

29. What is sorting and which common algorithms should I know?

Answer: Know bubble/insertion/selection for fundamentals, merge sort and quicksort for interviews, and heap sort. Merge sort is $O(n \log n)$ worst-case and stable; quicksort is $O(n \log n)$ average but $O(n^2)$ worst-case without suitable pivot behavior.

30. How should you approach a DSA interview problem?

Answer: Clarify inputs and constraints, give a brute-force idea, identify the bottleneck, choose the appropriate data structure/algorithm, state time and space complexity, then test edge cases such as empty input, duplicates, one element, and large values.

Part 2 — AI / LLM Interview Questions

RAG & Retrieval

RAG & Retrieval — 1. How would you design an effective chunking strategy?

Answer: Start from document structure rather than blindly using a fixed size. Keep headings, paragraphs, tables, and related sentences together; choose a token/character limit appropriate to the embedding model; use modest overlap when needed; and evaluate retrieval recall on real queries. For long documents, hierarchical or parent-child chunks can preserve context.

RAG & Retrieval — 2. What causes poor retrieval quality in RAG systems?

Answer: Common causes are bad chunk boundaries, weak embeddings, poor preprocessing, missing metadata, query-document vocabulary mismatch, noisy documents, unsuitable top-k, and an embedding model that does not fit the domain. Debug by measuring retrieval separately from generation.

RAG & Retrieval — 3. How would you improve semantic search accuracy?

Answer: Improve document cleaning and chunking, use a stronger domain-appropriate embedding model, normalize/filter metadata, tune top-k, use query rewriting when useful, combine dense and lexical retrieval, and add a reranker. Validate changes with a labeled retrieval benchmark.

RAG & Retrieval — 4. Difference between dense, sparse, and hybrid retrieval?

Answer: Dense retrieval uses vector embeddings and captures semantic similarity. Sparse retrieval, such as BM25, relies on term matching and is strong for exact words, IDs, and rare terms. Hybrid combines both, often improving robustness across semantic and exact-match queries.

RAG & Retrieval — 5. When should you use a reranker?

Answer: Use a reranker when the first-stage retriever returns reasonable candidates but their ordering is imperfect. Retrieve a larger candidate set cheaply, then use a cross-encoder or other reranker to score query-document pairs more accurately before sending the best passages to the LLM.

RAG & Retrieval — 6. How would you evaluate retrieval performance separately from generation?

Answer: Create query-answer-document ground truth and measure retrieval metrics such as Recall@k, Precision@k, MRR, and nDCG. Check whether the evidence needed to answer a question appears in the retrieved set before evaluating the LLM's final answer.

RAG & Retrieval — 7. How do metadata filters improve retrieval?

Answer: Metadata filters restrict search to relevant subsets such as product, language, department, date, or document version. This reduces noise and can improve both precision and latency, but incorrect or missing metadata can exclude the correct evidence.

RAG & Retrieval — 8. How would you update a RAG system with new documents in real time?

Answer: Use an ingestion pipeline: detect the new/changed document, parse and chunk it, generate embeddings, upsert vectors plus metadata, and atomically update the index/version. For deletes or updates, use stable document/chunk IDs so stale vectors can be removed or replaced.

RAG & Retrieval — 9. What are the trade-offs between larger and smaller embedding models?

Answer: Larger models can provide better semantic representations but require more memory, compute, latency, and cost. Smaller models are faster and cheaper. Choose based on retrieval quality on your domain benchmark, not model size alone.

RAG & Retrieval — 10. How would you debug an LLM that gives correct answers only sometimes?

Answer: Trace the whole pipeline: prompt, retrieved chunks, ranking, context length, model/version, temperature, tool calls, and output parsing. Reproduce failures with a fixed evaluation set, determine whether the issue is retrieval or generation, then change one component at a time.

Production LLM Engineering

Production LLM Engineering — 1. How would you deploy an LLM application to production?

Answer: Separate the application API from model serving, containerize components, use authentication and rate limits, manage secrets securely, add observability, health checks, autoscaling, caching where safe, and automated evaluation. Use staged deployment such as canary release before full rollout.

Production LLM Engineering — 2. How would you monitor hallucinations over time?

Answer: Maintain a production evaluation set, sample real interactions with privacy controls, measure groundedness/faithfulness and task-specific correctness, and track trends by model version, prompt version, and traffic segment. Human review is useful for high-risk cases.

Production LLM Engineering — 3. How would you design fallback mechanisms when an LLM fails?

Answer: Use timeouts, retries with bounded backoff, circuit breakers, and a fallback model/provider when appropriate. For deterministic tasks, fall back to rules or conventional services. If no safe answer is possible, return a clear failure instead of fabricating information.

Production LLM Engineering — 4. How would you securely handle confidential enterprise data?

Answer: Apply least-privilege access, encryption in transit and at rest, strong authentication, tenant isolation, secret management, audit logs, data minimization, retention controls, and appropriate model/provider data policies. Redact sensitive data where possible and prevent unauthorized retrieval.

Production LLM Engineering — 5. How would you implement caching for LLM responses?

Answer: Normalize/cache safe repeated requests using a key based on relevant model, prompt, parameters, and input. Use TTLs and invalidation when knowledge changes. Semantic caching can reuse answers for similar queries, but it needs similarity thresholds and careful handling of personalized or sensitive responses.

Production LLM Engineering — 6. How would you evaluate LLM quality after deployment?

Answer: Combine offline benchmark scores with online signals: task success, user feedback, groundedness, safety violations, latency, cost, and escalation rate. Compare model/prompt versions using controlled experiments and keep regression tests for important workflows.

Production LLM Engineering — 7. How would you reduce operational costs of an LLM application?

Answer: Reduce unnecessary tokens, use smaller models for simple tasks, cache repeated work, batch requests where possible, optimize prompts and retrieval, use quantization/self-hosting when economical, route requests by difficulty, and monitor cost per successful task rather than only cost per request.

Production LLM Engineering — 8. How would you handle sudden spikes in traffic?

Answer: Use a queue, rate limiting, autoscaling, load balancing, bounded concurrency, batching, and backpressure. Cache where possible and prioritize critical requests. Monitor queue depth, latency, GPU utilization, errors, and saturation.

Production LLM Engineering — 9. How would you safely roll out a new LLM version?

Answer: Run offline regression and safety tests first, then shadow or canary traffic, compare quality/cost/latency against the current version, and gradually increase traffic. Keep versioned prompts/configuration and an immediate rollback path.

Production LLM Engineering — 10. How would you troubleshoot an LLM application that became slower after deployment?

Answer: Break latency into components: queue time, retrieval, network, model time-to-first-token, generation time, tool calls, and post-processing. Compare metrics before/after the release and check token counts, model configuration, batching, GPU utilization, provider latency, and database/index performance.

Prompt Engineering

Prompt Engineering — 1. When would you use zero-shot, one-shot, and few-shot prompting?

Answer: Zero-shot is best when instructions are clear and no example is needed. One-shot gives one example to clarify the desired behavior. Few-shot gives multiple examples when the task has a nuanced format, classification boundary, or domain-specific pattern.

Prompt Engineering — 2. How do system prompts influence model behavior?

Answer: System instructions establish high-priority behavioral constraints, role, output requirements, and policies. They guide the model but are not a security boundary by themselves; application logic must still enforce permissions and validate outputs.

Prompt Engineering — 3. How would you reduce prompt injection risks?

Answer: Treat external content as untrusted data, clearly separate instructions from retrieved/user content, minimize tool permissions, validate tool arguments, enforce authorization outside the model, filter or isolate suspicious instructions, and require confirmation for high-impact actions.

Prompt Engineering — 4. What is prompt chaining and when is it useful?

Answer: Prompt chaining breaks a complex task into sequential steps, where the output of one step becomes input to another. It is useful when decomposition improves reliability, enables validation, or lets different prompts/models handle specialized subtasks.

Prompt Engineering — 5. How would you design prompts for structured JSON output?

Answer: Define the schema precisely, specify required fields and types, give a small valid example when helpful, and use native structured-output/function-calling features when available. Validate the result against a JSON schema and retry or repair safely on validation failure.

Prompt Engineering — 6. How do you handle prompts that exceed the context window?

Answer: Reduce irrelevant content, summarize or compress history, retrieve only relevant chunks, truncate strategically, and split the task into stages. For large documents, use hierarchical summarization or map-reduce style processing.

Prompt Engineering — 7. How would you evaluate prompt quality?

Answer: Use a fixed representative evaluation set with clear success criteria. Compare versions on correctness, format compliance, safety, latency, and cost. Include adversarial and edge cases and, when possible, blind human evaluation.

Prompt Engineering — 8. How would you reduce token usage without losing accuracy?

Answer: Remove redundant instructions, shorten examples, retrieve only relevant context, summarize long history, use structured compact formats, and route simple tasks to smaller models. Measure quality after each optimization.

Prompt Engineering — 9. What are the trade-offs between longer and shorter prompts?

Answer: Long prompts can provide more context and constraints but increase cost, latency, and the chance of distraction or conflicting instructions. Short prompts are cheaper and often clearer but may omit important context. The goal is sufficient, relevant context—not maximum length.

Prompt Engineering — 10. How would you version and manage prompts in production?

Answer: Store prompts as versioned artifacts in source control or a prompt registry, attach metadata such as model and configuration, test each version against an evaluation suite, and support rollback. Log which prompt version produced each production response.

LLM Inference & Optimization

LLM Inference & Optimization — 1. What is KV caching and why is it important?

Answer: During autoregressive generation, the model repeatedly needs attention keys and values for previous tokens. KV caching stores them so they do not have to be recomputed for every new token, greatly reducing repeated computation and improving decoding latency.

LLM Inference & Optimization — 2. How does quantization reduce inference cost?

Answer: Quantization represents model weights and sometimes activations with fewer bits, such as INT8 or lower precision instead of FP16/BF16. This reduces memory bandwidth and storage and can improve throughput, though aggressive quantization can reduce quality.

LLM Inference & Optimization — 3. What is continuous batching?

Answer: Continuous batching dynamically adds new requests to an active generation batch as other requests finish, instead of waiting for the whole batch. It improves GPU utilization and throughput for variable-length workloads.

LLM Inference & Optimization — 4. How would you reduce LLM latency?

Answer: Use a smaller or optimized model when quality permits, quantize, reduce input/output tokens, use KV caching, continuous batching, efficient inference engines, prompt/retrieval optimization, and low-latency networking. Measure time-to-first-token and time-per-output-token separately.

LLM Inference & Optimization — 5. What causes GPU memory bottlenecks during inference?

Answer: Major causes include model weights, KV cache growth with context length and batch size, activations/workspace, fragmentation, and concurrent requests. Long contexts and high concurrency can make KV cache a dominant memory consumer.

LLM Inference & Optimization — 6. How would you serve multiple LLMs efficiently?

Answer: Use model routing, shared infrastructure, dynamic loading where practical, batching per model, quantization, and separate resource pools for different latency/quality requirements. Route each request to the smallest model that meets its quality target.

LLM Inference & Optimization — 7. What is speculative decoding?

Answer: A smaller draft model proposes several tokens, and a larger target model verifies them in parallel. Accepted draft tokens reduce expensive target-model decoding steps, potentially improving generation speed without changing the target model's final distribution.

LLM Inference & Optimization — 8. How would you optimize throughput for high-traffic AI applications?

Answer: Increase effective GPU utilization using continuous batching, appropriate batch sizes, efficient kernels, quantization, request scheduling, caching, and horizontal scaling. Avoid optimizing throughput at the expense of unacceptable tail latency.

LLM Inference & Optimization — 9. What metrics would you monitor for LLM inference?

Answer: Monitor time-to-first-token, inter-token latency, end-to-end latency, throughput/tokens per second, request rate, error rate, queue time, GPU utilization/memory, token counts, cache hit rate, and cost. Track p50/p95/p99 latency for user experience.

LLM Inference & Optimization — 10. How would you choose between cloud-hosted and self-hosted LLMs?

Answer: Compare quality, total cost, latency, privacy/compliance, control, scaling needs, operational expertise, model customization, and vendor dependence. Cloud is often faster to start; self-hosting can make sense for high sustained volume, strict data control, or specialized models.

LLM Fundamentals

LLM Fundamentals — 1. How does an LLM generate the next token?

Answer: The tokenizer converts text into token IDs. A transformer processes the context and produces logits over the vocabulary. Softmax converts logits to probabilities, then a decoding strategy selects the next token. The token is appended and the process repeats autoregressively.

LLM Fundamentals — 2. Why are decoder-only models preferred for text generation?

Answer: Decoder-only transformers use causal self-attention, so each token can attend only to previous tokens. This matches autoregressive generation directly and scales naturally for next-token prediction and open-ended text generation.

LLM Fundamentals — 3. Explain the role of tokenization in LLMs.

Answer: Tokenization converts raw text into discrete subword/byte-level units that the model can process. It determines sequence length, vocabulary coverage, and therefore affects cost, context usage, and how efficiently different languages or symbols are represented.

LLM Fundamentals — 4. What happens during pretraining?

Answer: The model is trained on a very large corpus to learn statistical patterns and representations, commonly through next-token prediction. Gradient descent updates billions of parameters so the model becomes capable of language understanding and generation.

LLM Fundamentals — 5. How does instruction tuning improve model performance?

Answer: Instruction tuning trains the model on curated instruction-response examples so it learns to follow user requests, produce useful formats, and behave more consistently on tasks. It adapts a pretrained model toward instruction following rather than teaching language from scratch.

LLM Fundamentals — 6. What is the purpose of RLHF in LLM training?

Answer: RLHF uses human preference data to train a reward model or preference objective and then optimize the language model toward responses humans prefer. It can improve helpfulness and alignment, although it requires careful preference data and evaluation.

LLM Fundamentals — 7. How do embeddings differ from hidden states?

Answer: An embedding is a vector representation intended to represent an item such as a token, sentence, or document for comparison or downstream use. A hidden state is an intermediate representation produced inside the model for a particular layer and context. A hidden state can be used to derive embeddings but is not automatically the same thing.

LLM Fundamentals — 8. Why do LLMs hallucinate even with good prompts?

Answer: LLMs optimize likely token sequences rather than guaranteeing factual truth. Missing knowledge, ambiguous questions, weak retrieval, conflicting context, model limitations, and decoding randomness can all produce unsupported claims. Grounding, tool use, constrained outputs, and verification reduce risk.

LLM Fundamentals — 9. What factors affect an LLM's context window?

Answer: The model architecture and training determine the maximum supported sequence length. Effective usage is also constrained by memory, inference cost, attention/KV-cache requirements, tokenizer behavior, and the model/provider's configured limit.

LLM Fundamentals — 10. How would you choose the right LLM for a production application?

Answer: Define required tasks and quality first, then benchmark candidate models on representative data. Compare accuracy, structured-output reliability, latency, context needs, tool use, safety, privacy, cost, licensing, and operational requirements. Choose the smallest model that reliably meets the target.

Interview tip: For technical interviews, answer in this order when possible: definition → how it works → example/use case → complexity or trade-off → practical consideration.